# Authentification sur API Symfony

## Table des matières

1. Introduction ..... 2
2. Les Bundles ..... 2
3. L'Utilisateur ..... 2
4. Les clés SSL ..... 2
5. Security.yaml ..... 3
6. Routing ..... 4
7. Et notre User dans tout ça ? ..... 5
8. Tester et comprendre ce qu'on a mis en place ..... 5

## 1. Introduction

Toutes les API ne sont pas publiques, il en existe certaines pour lesquelles on est obligés de s'authentifier pour pouvoir les utiliser. Cette authentification correspond dans la plupart des cas à un **abonnement** souscrit au fournisseur de l'API.

Nous allons ici voir comment identifier un utilisateur pour utiliser une API en reprenant le projet Autopote, en quelques étapes simples ☺

## 2. Les Bundles

Comme d'habitude avec Symfony, on a la chance d'avoir des bundles prévus pour gérer les tâches communes qu'un développeur peut rencontrer. Ici on va en installer deux :

```
composer require symfony/security-bundle
```

```
composer require lexik/jwt-authentication-bundle
```

## 3. L'Utilisateur

Afin de pouvoir authentifier des Utilisateurs il nous faut… Des utilisateurs, c'est ça. On va donc utiliser le **MakerBundle** pour créer notre entité User, comme ceci :

```
php bin/console make:user
```

Plusieurs questions vont nous être posées, on l'a déjà vu dans le cours sur l'Authentification, nous allons répondre par défaut à chacune d'entre elles.

N'oubliez pas de faire votre migration et de migrer après avoir créé cette entité User :

```
php bin/console make:migration
```

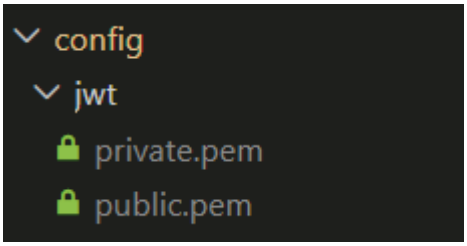
```
php bin/console doctrine:migrations:migrate
```

## 4. Les clés SSL

Pour chiffrer le token, Lexik utilise ce qu'on appelle une **paire** de clés **RSA**. Ce sont deux codes générés qui vont servir de clé de cryptage. On les génère automatiquement grâce au Bundle qu'on a installé en étape 2 :

```
php bin/console lexik:jwt:generate-keypair
```

Deux fichiers vont être générés ici :




*IL EST INDISPENSABLE QUE CES DEUX FICHIERS SOIENT .GITIGNORÉS. Vérifiez bien qu'ils sont grisés, si ils ne le sont pas ils faut les rajouter dans votre .gitignore*

Ces chemins ont également été rajoutés dans le fichier .env :

```
42
43  ###> lexik/jwt-authentication-bundle ###
44  JWT_SECRET_KEY=%kernel.project_dir%/config/jwt/private.pem
45  JWT_PUBLIC_KEY=%kernel.project_dir%/config/jwt/public.pem
46  JWT_PASSPHRASE=f5fe62687fb861b1af27bff8321ce97619e04576ec63d634b6acf78a33fa47ee
47  ###< lexik/jwt-authentication-bundle ###
48
```

## 5. Security.yaml

Dans le fichier security.yaml, on va devoir lui préciser également les chemins vers les clés à utiliser. On va globalement modifier notre fichier security.yaml pour qu'il ressemble à ceci, avec également les chemins vers les endpoints de login :

```yaml
security:
    password_hashers:
        App\Entity\User:
            algorithm: auto

    providers:
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email
```

```yaml
firewalls:
    dev:
        pattern: ^/(_(profiler|wdt)|css|images|js)/
        security: false

    login:
        pattern: ^/api/login
        stateless: true
        json_login:
            check_path: /api/login
            username_path: email
            password_path: password
            success_handler: lexik_jwt_authentication.handler.authentication_success
            failure_handler: lexik_jwt_authentication.handler.authentication_failure

    api:
        pattern: ^/api
        stateless: true
        jwt: ~

access_control:
    - { path: ^/api/login, roles: PUBLIC_ACCESS }
    - { path: ^/api/docs, roles: PUBLIC_ACCESS }
    - { path: ^/api, roles: IS_AUTHENTICATED_FULLY }
```

Le firewall **login** intercepte uniquement POST /api/login et délègue la vérification du mot de passe et la génération du token à Lexik

Le firewall **api** protège toutes les autres routes /api/* en vérifiant la présence et la validité du token JWT dans le header Authorization: Bearer <token>.

L'**access_control** affine les règles : /api/login et /api/docs restent publics, tout le reste exige d'être authentifié.

## 6. Routing

On va également rajouter une précision dans /config/routes.yaml :

```yaml
controllers:
    resource: routing.controllers

api_login:
    path: /api/login
    methods: [POST]
```

## 7. Et notre User dans tout ça ?

On va avoir besoin d'un User pour tester. Rajoutons-le donc dans notre Fixture :

```
use Symfony\Component\PasswordHasher\Hasher\UserPasswordHasherInterface;

class AppFixtures extends Fixture
{
    public function __construct(
        private UserPasswordHasherInterface $hasher
    ) {}
```

Premièrement on met le **Use** et on injecte le Hasher de mot de passe pour gérer le cryptage du mot de passe

Puis dans la fonction **load()**, au dessus du reste, on rajoute ceci :

NE PAS OUBLIER LE USE POUR LA CLASSE USER()

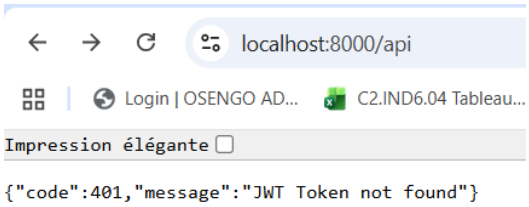
```
class AppFixtures extends Fixture
{
    public function __construct(
        private UserPasswordHasherInterface $hasher
    ) {}

    public function load(ObjectManager $manager): void
    {

        // Création de l'utilisateur de test
        $user = new User();
        $user->setEmail('admin@autopote.fr');
        $user->setPassword($this->hasher->hashPassword($user, 'password123'));
        $user->setRoles(['ROLE_ADMIN']);
        $manager->persist($user);
```

On recharge la fixture :

```
php bin/console doctrine:fixtures:load
```

## 8. Tester et comprendre ce qu'on a mis en place

L'url https://localhost:8000/api vous renvoie ceci :




Qu'est-ce qu'on a cassé ? Rien factuellement, Symfony ne fait qu'obéir aux règles que l'on a établi dans **security.yaml**

- L'api est maintenant verrouillée derrière l'obtention d'un JWT
- La doc a été déplacée vers **/api/docs**
- Le login se fait sur l'endpoint **/api/login**

Il ne nous reste plus qu'à ouvrir les portes aux autres domaines potentiels en modifiant le fichier nelmio_cors.yaml :

```
nelmio_cors:
    defaults:
        origin_regex: true
        allow_origin: ['%env(CORS_ALLOW_ORIGIN)%']
        allow_methods: ['GET', 'OPTIONS', 'POST', 'PUT', 'PATCH', 'DELETE']
        allow_headers: ['Content-Type', 'Authorization']
        expose_headers: ['Link']
        max_age: 3600
    paths:
        '^/api/':
            allow_origin: ['http://localhost:4200']
            allow_headers: ['Authorization']
            allow_methods: ['POST', 'PUT', 'GET', 'DELETE', 'OPTIONS']
            max_age: 3600
```

# EXERCICE

1 – Dans l'appli Angular Autopote, obtenez le JWT de l'API en requêtant l'endpoint **/api/login**.

ATTENTION , CA SERA UNE REQUETE EN POST ☺

Passez lui ceci dans le **body** de la requête : { "email": "admin@autopote.fr", "password": "password123" }. Vous devriez avoir une réponse comme ceci :

{

"token": "eyJ0eXAiOiJKV1QiLCJhbGci..."

}

2 – Stockez ce Token dans le **localStorage** du navigateur à la réception

3 - Testez de requêter un autre endpoint de l'API, au hasard **/api/parts** en lui passant dans les headers ceci : Authorization: Bearer <token>

4 – Une fois que vous avez mis tout ça en place et que tout fonctionne, mettez en place un Interceptor ☺